

Experiment – 2

Aim: Implementation and Time analysis of linear and binary search algorithm.

Hardware Required:

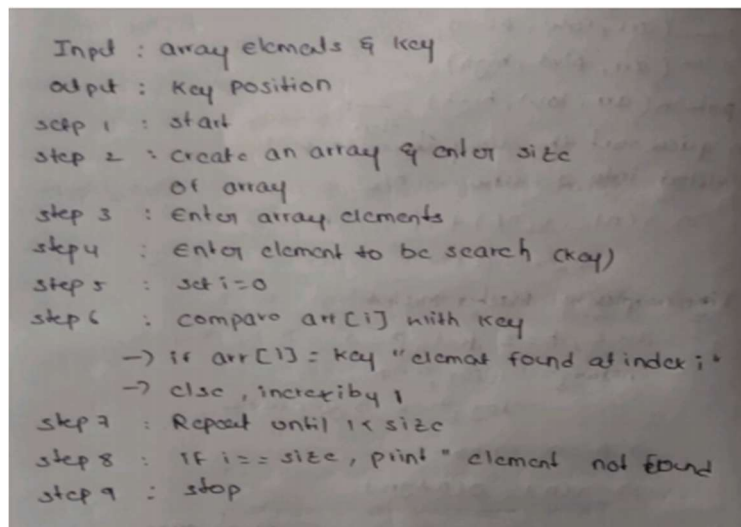
Mouse, keyboard.

Software Required:

code blocks.

a) Linear Search

Algorithm:



```
Input : array elements & key
Output : Key position
step 1 : start
step 2 : create an array & enter size
         of array
step 3 : Enter array elements
step 4 : Enter element to be search (key)
step 5 : set i = 0
step 6 : compare arr[i] with key
         -> if arr[i] = key "element found at index i"
         -> else, increment by 1
step 7 : Repeat until i < size
step 8 : If i == size, print "element not found"
step 9 : stop
```

Programm:

```
#include<iostream>
using namespace std;

class linearsearch {
public:
    int arr[100], size, key;

    void arrayinput() {
        cout << "enter size of array: ";
        cin >> size;
        cout << "enter array elements: ";
        int i;
        for(i = 0; i < size; i++) {
            cin >> arr[i];
        }
    }

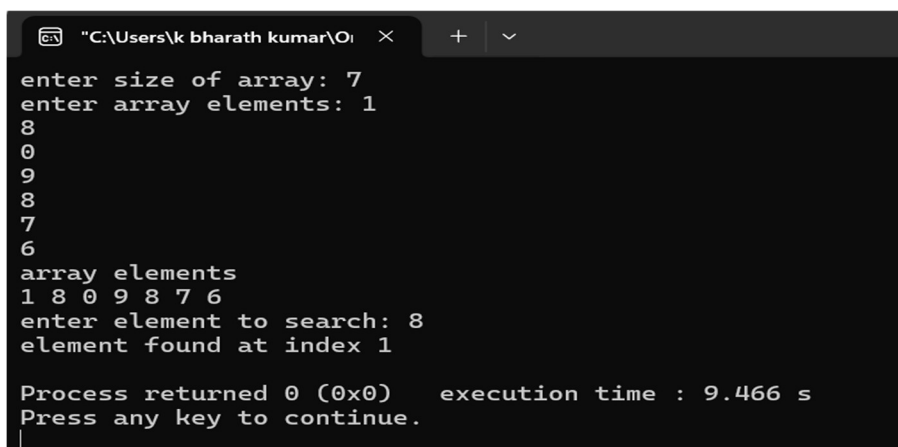
    void printarray() {
        int i;
        for(i = 0; i < size; i++) {
            cout << arr[i] << " ";
        }
        cout << endl;
    }

    void searchlogic() {
        int i;
        for(i = 0; i < size; i++) {
            if(arr[i] == key) {
                cout << "element found at index " << i << endl;
                break;
            }
        }
        if(i == size) {
            cout << "element not found" << endl;
        }
    }
};

int main()
{
    linearsearch ls;
    ls.arrayinput();
```

```
cout << "array elements" << endl;
ls.printarray();
cout << "enter element to search: ";
cin >> ls.key;
ls.searchlogic();
return 0;
}
```

Output:



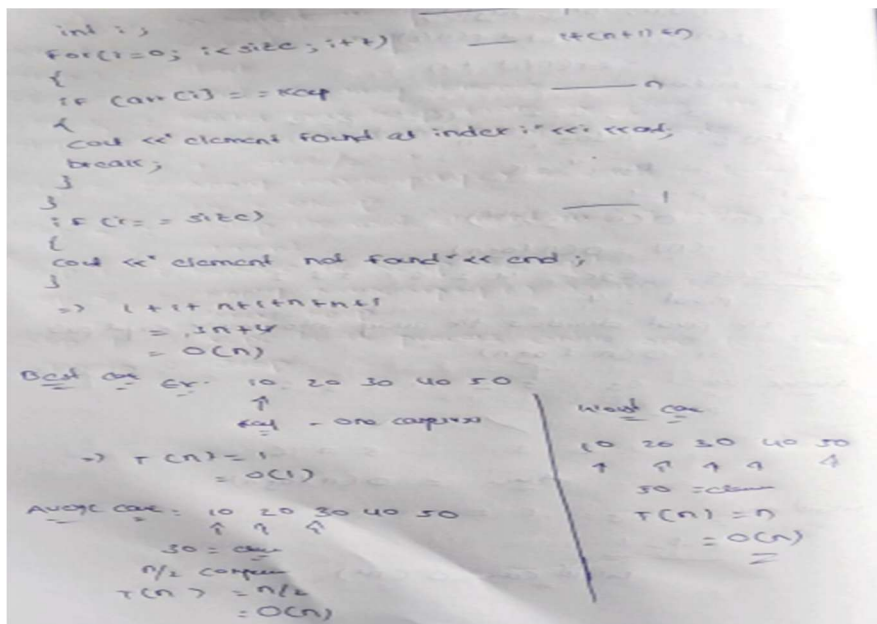
```

C:\Users\k bharath kumar\O...
enter size of array: 7
enter array elements: 1
8
0
9
8
7
6
array elements
1 8 0 9 8 7 6
enter element to search: 8
element found at index 1

Process returned 0 (0x0)   execution time : 9.466 s
Press any key to continue.

```

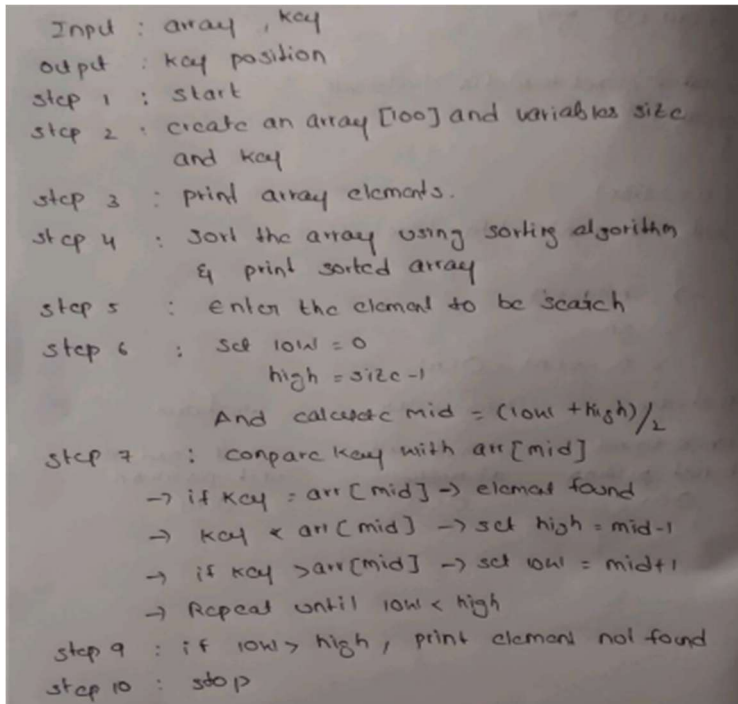
Time complexity:



$\text{int } i;$
 $\text{for}(i=0; i < \text{size}; i++)$ — $\text{if}(n+1)$
 $\{$
 $\text{if } (\text{arr}[i]) == \text{key}$ — n
 $\{$
 $\text{cout} << \text{"element found at index " } i << \text{"\n"};$
 $\text{break};$
 $\}$
 $\}$
 $\text{if } (i == \text{size})$ — 1
 $\{$
 $\text{cout} << \text{"element not found"} << \text{"\n"};$
 $\}$
 $\Rightarrow 1 + 1 + n + 1 + 1 + 1$
 $= 5n + 4$
 $= O(n)$
Best case Ex: 10 20 30 40 50
 $\uparrow$
 $\text{key} = \text{one comparison}$
 $\Rightarrow T(n) = 1$
 $= O(1)$
Average case: 10 20 30 40 50
 $\uparrow \uparrow \uparrow$
 $30 = \text{key}$
 $n/2$ comparison
 $T(n) = n/2$
 $= O(n)$
Worst case
10 20 30 40 50
 $\uparrow \uparrow \uparrow \uparrow \uparrow$
 $50 = \text{key}$
 $T(n) = n$
 $= O(n)$

b) Binary Search

Algorithm:



```
Input : array , key
Output : key position
step 1 : start
step 2 : create an array [100] and variables size
         and key
step 3 : print array elements.
step 4 : Sort the array using sorting algorithm
         & print sorted array
step 5 : Enter the element to be search
step 6 : set low = 0
         high = size-1
         And calculate mid = (low + high) / 2
step 7 : compare key with arr[mid]
         -> if key = arr[mid] -> element found
         -> key < arr[mid] -> set high = mid-1
         -> if key > arr[mid] -> set low = mid+1
         -> Repeat until low < high
step 8 : if low > high , print element not found
step 9 : stop
```

Program:

```
#include<iostream>
using namespace std;

class binarysearch
{
public:

int arr[100],size,key;

void arrayinput()
{
    cout<<"enter size of array: ";
    cin>>size;
```

```
cout<<"enter array elements: ";

int i;
for(i=0;i<size;i++)
{
    cin>>arr[i];
}
}

void printarray()
{
    int i;
    for(i=0;i<size;i++)
    {
        cout<<arr[i]<<" ";
    }
    cout<<endl;
}

void bubblelogic()
{
    int i,j,temp;

    for(i=0;i<size-1;i++)
    {
        for(j=0;j<size-i-1;j++)
        {
            if(arr[j]>arr[j+1])
            {
                temp=arr[j];
                arr[j]=arr[j+1];
                arr[j+1]=temp;
            }
        }
    }
}

void searchlogic()
{
    int low,high,mid;

    low=0;
    high=size-1;
```

```
while(low<=high)
{
    mid=(low+high)/2;

    if(key==arr[mid])
    {
        cout<<"element found";
        return;
    }
    else if(key<arr[mid])
    {
        high=mid-1;
    }
    else
    {
        low=mid+1;
    }
}

cout<<"element not found";
}
};

int main()
{
    binarysearch bs;

    bs.arrayinput();

    bs.bubblelogic();

    cout<<"sorted array"<<endl;
    bs.printarray();

    cout<<"enter element to search: ";
    cin>>bs.key;

    bs.searchlogic();

    return 0;
}
```

Output:

```
"C:\Users\k bharath kumar\O... x + v
enter size of array: 6
enter array elements: 7
7
8
2

4
5
sorted array
2 4 5 7 7 8
enter element to search: 6
element not found
Process returned 0 (0x0)   execution time : 13.031 s
Press any key to continue.
|
```

Time complexity:

Binary search

while (low <= high) $\rightarrow \log n + 1$

$\Rightarrow$ { mid = (low + high) / 2 $\rightarrow \log n$

if key == arr[mid] $\rightarrow \log n$

$\Rightarrow$ (key < arr[mid]) $\rightarrow \log n$

$\Rightarrow$ high = mid - 1; } $\rightarrow \log n$

low = mid + 1

$\Rightarrow 5 \log n$

$= \log n \rightarrow T(n) = O(\log n)$

Best case:-

eg: 10 20 30 40 50 60 70

element = 40

First mid

$T(n) = 1$

$= O(1)$

Worst case: 10 20 30 40 50 60 70

element = 60

$= n \rightarrow n/2 \rightarrow n/4 \rightarrow n/8$

$= 2^{k-1} = n$

$k = \log_2 n \rightarrow O(\log n)$

Worst case: Element not present

$\Rightarrow n \rightarrow n/2 \rightarrow n/4 \rightarrow n/8 \dots$

$\Rightarrow O(\log n)$

Conclusion:

As per the experiment to Implementation and Time analysis of linear and binary search algorithm., Output was verified successfully according to the Experiment.